SENTINEL

Setting up the machine you build on

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

What this covers

You build and run SENTINEL on Linux. This guide sets up a Linux system on your computer, and it installs the tools that all four labs use. Then it makes your own copy of the course repository. Do this one time, for all four labs.

Read the section for your own computer, then read **Every machine**, which applies to all of them.

A warning before you start, because it can cost marks. On Windows, keep your work in the Linux file system, not on the Windows drive. Section 3 states where, and why the acceptance test[1] fails when the work is in the wrong place.

Which processor you need

Both processors in the table work. The course does not prefer one of them.

Your computer	What you install	`uname -m` says
Mac, Apple silicon (M1 to M4)	Ubuntu for `aarch64`, in a virtual machine	`aarch64`
Mac, Intel	Ubuntu for `x86_64`, in a virtual machine	`x86_64`
Windows	Ubuntu through WSL2	`x86_64`
Linux	nothing. You already have it	either

1 Mac with Apple silicon

An Apple silicon Mac has an `aarch64` processor, so it runs `aarch64` Linux at full speed in a virtual machine. Use UTM, which is free.

1. Install UTM from https://mac.getutm.app/.
2. Download the Ubuntu Server **ARM64** image from https://ubuntu.com/download/server/arm. Take the current long term support release.
3. In UTM choose **Create a New Virtual Machine**, then **Virtualize**, then **Linux**. Select the image you downloaded.
4. Give the machine 4 GB of memory, 2 processors and 32 GB of disk. Less than 2 GB is not enough to build and run the labs together.
5. Start the machine and complete the Ubuntu installer. Choose to install OpenSSH when the installer offers it, because it lets you work from the Mac terminal.

[1] The acceptance test of Lab 1, `make accept1`, runs 34 checks of your library and your demo. The Lab 1 Brief, Section 4.3, `docs/lab1/lab1_brief.pdf`, describes it.

Choose **Virtualize** and not **Emulate**. Virtualize runs the Linux code directly on your processor. Emulate translates every instruction, and the labs become too slow to measure.

2 Mac with an Intel processor

The steps are the same as Section 1, with two differences. Download the Ubuntu Server **AMD64** image instead of ARM64, and `uname -m` then reports `x86_64`.

VirtualBox from https://www.virtualbox.org/ also works on an Intel Mac.

3 Windows

Windows runs Linux through WSL2, the Windows Subsystem for Linux. You do not need a virtual machine.

1. Open PowerShell as Administrator and run:

   ```
   wsl --install -d Ubuntu
   ```

2. Restart the computer when it asks.

3. Start **Ubuntu** from the Start menu. It asks for a user name and a password the first time. That password is for Linux, and it is not your Windows password.

4. Check that you have version 2, which the labs need:

   ```
   wsl -l -v
   ```

   The `VERSION` column must say `2`. If it says `1`, convert it:

   ```
   wsl --set-version Ubuntu 2
   ```

3.1 Where your work must live

Keep your repository in the Linux home directory. Do not keep it on the Windows drive.

In Ubuntu, your Linux home directory is `~`, which is `/home/<your name>`. The Windows drives are under `/mnt/c`, `/mnt/d` and the letters after them. You can reach both, but only one of them works.

Path	Use it?
`~/sentinel-lab`	**Yes.** A real Linux file system
`/mnt/c/Users/you/sentinel-lab`	**No.** A Windows drive

Why it matters. Lab 1 creates the mission log[2] with the permissions `0600`: the owner can read and write it, and nobody else can. It is a surveillance record. The acceptance test[3] checks that the permissions are exactly `0600`, and it checks another file for exactly `0644`.

[2] The mission log is the file in which SENTINEL records each action, with its time (requirement R6). The Lab 1 Brief, Section 1, `docs/lab1/lab1_brief.pdf`, states what Lab 1 writes to it.

[3] The acceptance test of Lab 1, `make accept1`, runs 34 checks of your library and your demo. The Lab 1 Brief, Section 4.3, `docs/lab1/lab1_brief.pdf`, describes it.

A Windows drive does not store Unix permissions. Your program asks for `0600`, the Windows drive cannot record that, and the file reports something else. The acceptance test[4] then reports a failure in code that is correct, and no message names the file system as the cause.

In `~`, this fault cannot occur.

To open your Linux files from Windows, type `\\wsl$` in File Explorer, or run `explorer.exe .` from the Ubuntu terminal.

4 Linux

Linux needs no step here. Go to **Every machine**.

Every machine

Each step after this point runs in Linux, on each type of computer.

Check the processor

```
uname -m
```

It prints `aarch64` or `x86_64`. The course gives its programs for both processors, and the answer tells you which ones your copy uses.

Install the tools

```
sudo apt update
sudo apt install build-essential python3 valgrind git zip manpages-dev procps
sudo apt install manpages-posix-dev
```

Tool	What the labs use it for
`gcc`, `make`	compile and link your library, your tests and your demonstration program
`python3`	serve the operations centre display to your browser[5]
`valgrind`	find memory faults and leaks in your code
`git`	receive each lab when the course releases it
`zip`	build the file you submit
`manpages-dev`	the manual pages for the system calls, which the prelab[6] asks you to read
`manpages-posix-dev`	the POSIX manual pages, such as `pthread_mutex_lock`, which Lab 4 uses
`pgrep`, `watch`	find the pid of your program, and read its `/proc` files (`procps`)

Check that the compiler answers:

[4] The acceptance test of Lab 1, `make accept1`, runs 34 checks of your library and your demo. The Lab 1 Brief, Section 4.3, `docs/lab1/lab1_brief.pdf`, describes it.

[5] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

[6] The Lab 1 prelab, `docs/lab1/prelab1.pdf` in your repository. It is due before the Lab 1 session.

```
gcc --version
make --version
```

Make your own copy of the repository

You do not work in the course repository. You make your own copy of it, you keep the copy private, and you keep it for all four labs. Your copy holds your solutions, so it must be private. The course gives you the address of the course repository.

1. Make your copy, one time. On GitHub, open the course repository, select **Use this template**, then **Create a new repository**, and set it to **Private**. On GitLab, open the course repository, select **Fork**, and set the visibility to **Private**.

   On GitHub, do **not** use Fork. A fork of a public repository is public, and GitHub cannot make a fork private.

2. Clone your copy, and connect it to the course repository, one time:

   ```
   cd ~
   git clone <your repository>
   cd sentinel-lab
   git remote add upstream <the course repository>
   git remote -v
   ```

   `origin` is your copy, and `upstream` is the course repository.

3. When the course releases a lab, get it:

   ```
   git pull upstream main
   ```

   This gets the files of the new lab, and each correction to the given files. Do it before you start each lab, and again when the course announces a correction.

You do not hand in this repository. The Submission Guide, `docs/submit/submit.pdf` in your copy, tells you how to hand in a lab.

Check that a build works

In your copy, build the given code:

```
cd ~/sentinel-lab
make
```

Before you write a file of a lab, `make` ends with these lines, and they are correct:

```
src/libsyscall: no Makefile yet
src/libproc: no Makefile yet
src/libipc: no Makefile yet
src/libsync: no Makefile yet
src/sentinel: no Makefile yet
```

`make` builds the given code, copies the given programs into `build/`, and then builds each lab that has a `Makefile`. No lab has one yet, so `make` names each of the five, and it stops with no error. When you see these lines, the machine is ready.

Reach the display in your browser

The labs show an operations centre display, the page.[7] Start it with:

```
make watch
```

Then open `http://localhost:8080/` in a browser. How you reach it depends on how you installed Linux.

Where you run Linux	How to open the display
WSL2 on Windows	Use any Windows browser. WSL2 forwards `localhost` for you
A virtual machine	Use a browser inside the virtual machine. To use the browser on the Mac instead, forward the port over SSH with `ssh -L 8080:localhost:8080 <user>@<vm>`
Linux	Use any browser on the machine

If you work over SSH

`web/README.md`, in your copy of the repository, tells you how to reach the page over SSH. `make watch` also prints the two ways when it finds an SSH connection.

If something does not work

What you see	What to do
`gcc: command not found`	`build-essential` is not installed. Run the install command again
The acceptance test[8] reports wrong file permissions	On Windows, your repository is on a Windows drive. Move it to `~`. See Section 3
A manual page is not there	Install `manpages-dev` and `manpages-posix-dev`. If `apt` cannot find `manpages-posix-dev`, run `sudo add-apt-repository multiverse`, then `sudo apt update`. Each starter file of a lab also gives the web address of each page
The browser cannot reach `localhost:8080`	`make watch` must continue to run in its own terminal. In a virtual machine, forward the port or use the browser inside the machine
The build or the labs run very slowly	In UTM, you created the machine with **Emulate**. Create it again with **Virtualize**

[7] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

[8] The acceptance test of Lab 1, `make accept1`, runs 34 checks of your library and your demo. The Lab 1 Brief, Section 4.3, `docs/lab1/lab1_brief.pdf`, describes it.